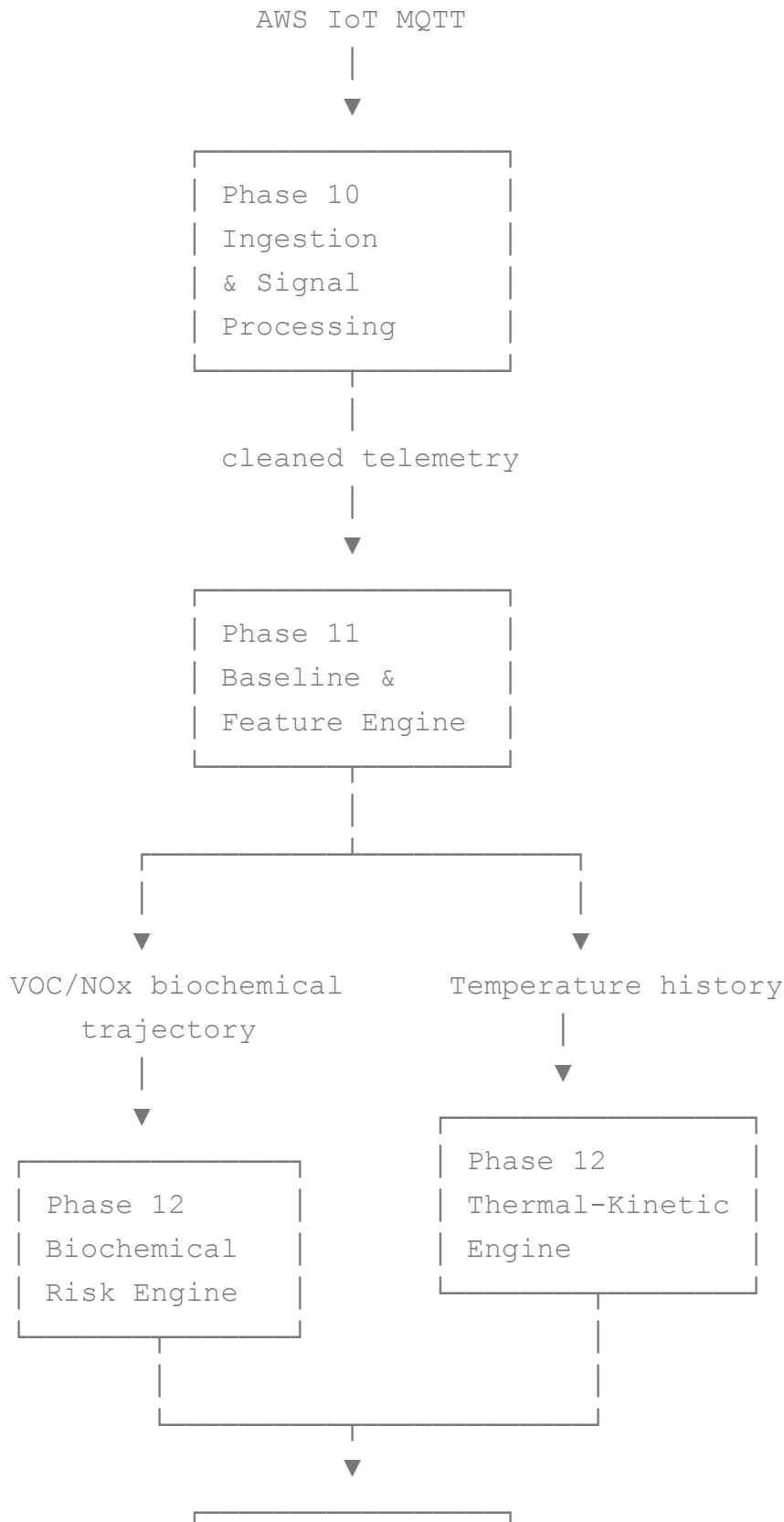
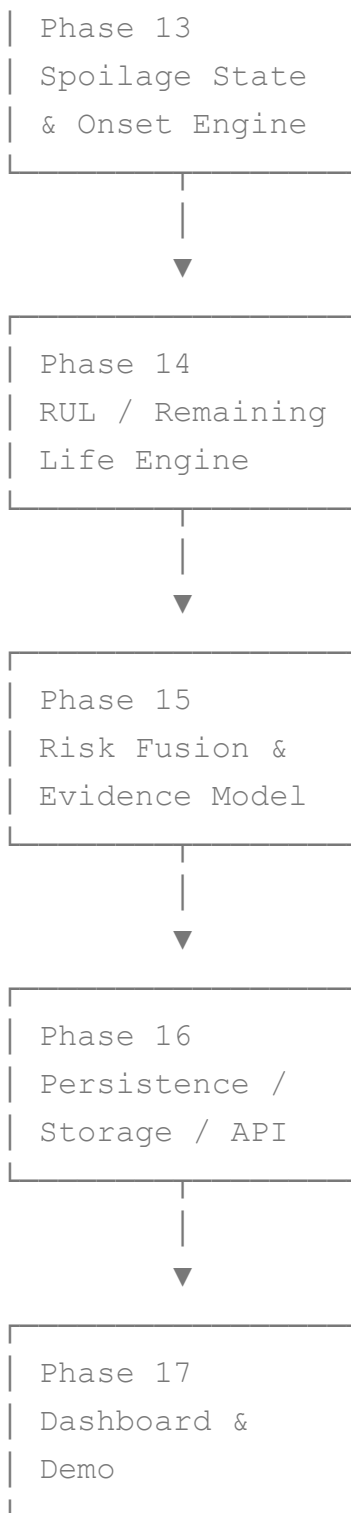


Axios – Production Roadmap

Overall architecture





Phase 10 – IoT Telemetry Ingestion

Goal

Turn the AWS IoT MQTT stream into a clean, timestamped internal telemetry stream.

Input

```
{
  "device_id": "FreshTrace-Node-01",
  "food_type": "paneer",
  "temperature_c": 30.20,
  "humidity_pct": 66.17,
  "pressure_hpa": 947.07,
  "voc_raw": 32213,
  "nox_raw": 21986,
  "voc_index": 0,
  "nox_index": 0,
  "wifi_rssi": -35
}
```

Processing

1. MQTT subscription.
2. JSON validation.
3. Timestamp attachment.
4. Device identification.
5. Food-type identification.
6. Numeric range validation.
7. Missing-value handling.
8. Duplicate/out-of-order detection.
9. Sensor sanity checks.
10. Local buffering if AWS/network connection disappears.

Internal representation

```
TelemetryReading(
    timestamp,
    device_id,
    food_type,
    temperature_c,
    humidity_pct,
    pressure_hpa,
    voc_raw,
    nox_raw,
    voc_index,
    nox_index,
```

```
wifi_rssi,  
)
```

Output

```
data/  
└─ telemetry/  
    └─ FreshTrace-Node-01/  
        └─ session_*.jsonl
```

Validation

Before continuing:

- timestamps monotonic
 - no impossible temperatures
 - no NaN/invalid sensor values
 - VOC/NOx readings numeric
 - device identity stable
 - sampling interval measured
-

Phase 11 – Signal Processing & Session Baseline

This becomes the foundation of everything downstream.

11.1 Session initialization

Every shipment/sample gets a session:

```
SESSION START  
  |  
  ▼  
collect baseline readings  
  |  
  ▼
```

baseline established



monitor degradation

The baseline is **specific to that device/session**, not a global constant.

For example:

```
baseline.voc
baseline.nox
baseline.temperature
baseline.humidity
```

11.2 Noise filtering

Raw sensor data will fluctuate.

We implement:

- moving median
- exponential moving average
- optional low-pass filter
- outlier rejection
- rate-of-change limits

We preserve both:

```
raw_value
filtered_value
```

Never overwrite the raw telemetry.

11.3 Derivatives

For VOC and NOx:

\$\$

$$v(t) = \frac{x_t - x_{t-1}}{\Delta t}$$

\$\$

and:

\$\$

$$a(t) = \frac{v_t - v_{t-1}}{\Delta t}$$

\$\$

giving:

```
VOC velocity  
VOC acceleration
```

```
NOx velocity  
NOx acceleration
```

This gives us the difference between:

```
high VOC
```

and:

```
VOC rapidly increasing
```

which is much more useful for onset detection.

11.4 Relative deviation

Instead of comparing devices using absolute raw ticks:

\$\$

$$D(t) = \frac{x(t) - x_0}{|x_0| + \epsilon}$$

\$\$

We maintain:

```
voc_relative_change  
nox_relative_change
```

and optionally running-range-normalized values.

Phase 12 – Biochemical Degradation Engine

This is the core SGP41-derived signal engine.

We explicitly **do not call this ML**.

It converts VOC/NOx trajectories into a biochemical degradation index.

12.1 VOC signal

Calculate:

```
VOC level deviation
VOC slope
VOC acceleration
VOC volatility
VOC persistence
```

12.2 NOx signal

Same:

```
NOx level deviation
NOx slope
NOx acceleration
NOx volatility
NOx persistence
```

12.3 Multi-signal biochemical score

Conceptually:

```
$$
B(t)=f(
D_{VOC},
D_{NOx},
S_{VOC},
S_{NOx},
```

A_{VOC},
A_{NOx},
P
)
\$\$

where:

- (D)= deviation
- (S)= slope
- (A)= acceleration
- (P)= persistence

Output:

```
{  
  "biochemical_risk": 0.64,  
  "voc_signal": 0.71,  
  "nox_signal": 0.52,  
  "degradation_velocity": 0.038  
}
```

Initially this is a **risk score**, not a statistically calibrated probability.

Phase 13 – Thermal-Kinetic Engine

This handles the other side of the problem.

The Product X specification explicitly calls for **Arrhenius-based thermal-kinetic exposure modeling**.

13.1 Temperature history

Maintain:

```
current temperature  
average temperature  
maximum temperature  
time above threshold  
temperature excursion duration
```

But we don't reduce the whole history to one maximum.

13.2 Arrhenius relative rate

Use:

$$k(T) = A e^{-E_a/(RT)}$$

For relative acceleration against a reference temperature:

$$\frac{k(T)}{k(T_{\text{ref}})}$$

$$e^{-\frac{E_a}{R} \left(\frac{1}{T} - \frac{1}{T_{\text{ref}}} \right)}$$

with temperature converted to Kelvin.

This produces a **relative thermal reaction-rate multiplier**.

13.3 Cumulative thermal exposure

Integrate over time:

$$E_{\text{thermal}}$$

$$\int k_{\text{relative}}(T(t)) dt$$

This becomes our cumulative thermal burden.

Output:

```
{
  "thermal_risk": 0.72,
  "thermal_exposure": 14.3,
  "temperature_excursion": true,
  "max_temperature_c": 34.8
}
```

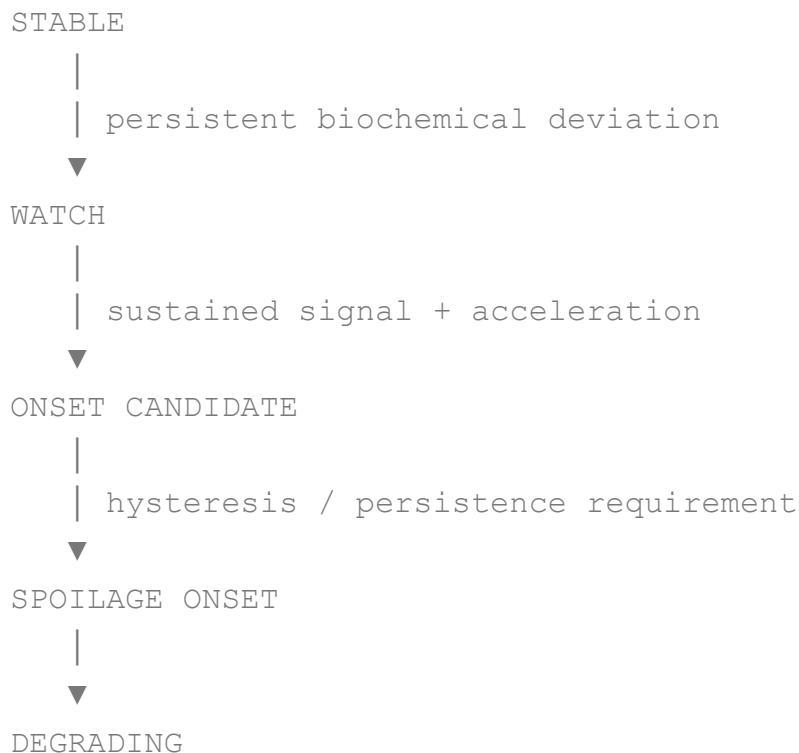
The actual normalization and food-specific parameters must be configurable rather than arbitrarily hardcoded.

Phase 14 – Spoilage Onset Detector

This answers:

When did degradation actually start?

This should be a state machine rather than a single threshold.



Requirements

A single noisy spike must **not** trigger spoilage.

We use:

- threshold
- persistence window
- hysteresis
- slope requirement
- minimum signal-to-noise ratio

Output:

```
{  
  "onset_detected": true,  
  "onset_timestamp": "2026-09-19T18:42:13Z",  
  "onset_confidence": 0.86  
}
```

Again, this is an engineered confidence score until experimentally calibrated.

Phase 15 – Remaining Useful Life Engine

This is the most delicate component.

We cannot simply say:

VOC = X → 4.3 days remaining

because we don't have a validated biological endpoint.

Instead:

```
current degradation state  
    +  
degradation velocity  
    +  
thermal exposure  
    ↓  
estimated time to risk threshold
```

Concept

If:

\$\$

$D(t)$

\$\$

is degradation state and:

\$\$

$\frac{dD}{dt}$

\$\$

is degradation velocity, then approximately:

\$\$

$T_{\text{remaining}}$

$\approx$

$\frac{D_{\text{critical}} - D(t)}{dD/dt}$

\$\$

But we should improve this using the observed trajectory rather than relying on one derivative.

Output

```
{
  "remaining_useful_life_hours": 38.4,
  "lower_bound_hours": 27.1,
  "upper_bound_hours": 55.8,
  "endpoint": "configured_risk_threshold"
}
```

The dashboard should explicitly call this:

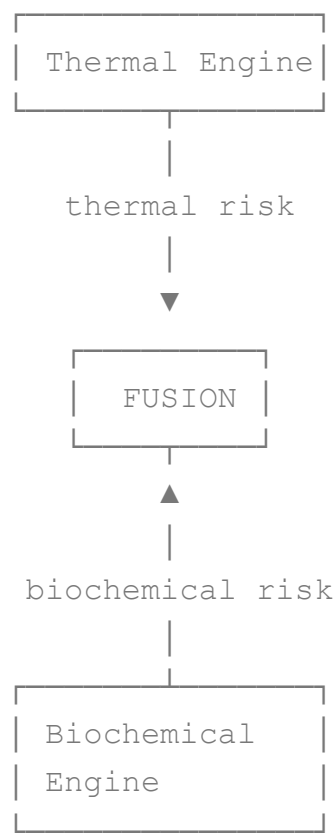
Estimated Remaining Useful Life

not scientifically validated shelf life.

Phase 16 – Thermal vs Biochemical Risk

Fusion

Now we combine the two independent mechanisms.



Output:

```
{
  "thermal_risk": 0.73,
  "biochemical_risk": 0.81,
  "overall_risk": 0.78,
  "dominant_risk": "biochemical"
}
```

Possible states:

```
THERMAL_DOMINANT
BIOCHEMICAL_DOMINANT
DUAL_RISK
```

LOW_RISK
INSUFFICIENT_DATA

This is particularly useful for the dispute/evidence use case described in the Product X material: the system should retain evidence about **when** the condition changed and whether the evidence points toward thermal exposure or biochemical degradation.

Phase 17 – Evidence & Event Engine

This is where Axios becomes more than a sensor dashboard.

Every important event gets recorded:

```
SESSION_STARTED  
BASELINE_ESTABLISHED  
TEMPERATURE_EXCURSION  
BIOCHEMICAL_ACCELERATION  
SPOILAGE_ONSET_CANDIDATE  
SPOILAGE_ONSET_CONFIRMED  
THERMAL_RISK_HIGH  
BIOCHEMICAL_RISK_HIGH  
RUL_THRESHOLD_CROSSED
```

Each event:

```
{  
  "event": "BIOCHEMICAL_ACCELERATION",  
  "timestamp": "...",  
  "severity": 0.82,  
  "evidence": {  
    "voc_slope": "...",  
    "nox_slope": "...",  
    "persistence_seconds": "...",  
    "temperature": "..."  
  }  
}
```

This gives us an auditable timeline rather than just a current number.

Phase 18 – API & Storage

Backend endpoints:

```
GET /devices
GET /devices/{device_id}/status
GET /devices/{device_id}/telemetry
GET /devices/{device_id}/risk
GET /devices/{device_id}/events
GET /devices/{device_id}/timeline
GET /devices/{device_id}/report
```

The primary state object could be:

```
{
  "device_id": "FreshTrace-Node-01",
  "food_type": "paneer",

  "state": "DEGRADING",

  "spoilage_risk": 0.74,

  "spoilage_onset": {
    "detected": true,
    "timestamp": "...",
    "confidence": 0.86
  },

  "remaining_useful_life": {
    "hours": 38.4,
    "lower": 27.1,
    "upper": 55.8
  },

  "degradation_velocity": 0.038,

  "risk": {
    "thermal": 0.42,
    "biochemical": 0.81,
    "dominant": "biochemical"
```

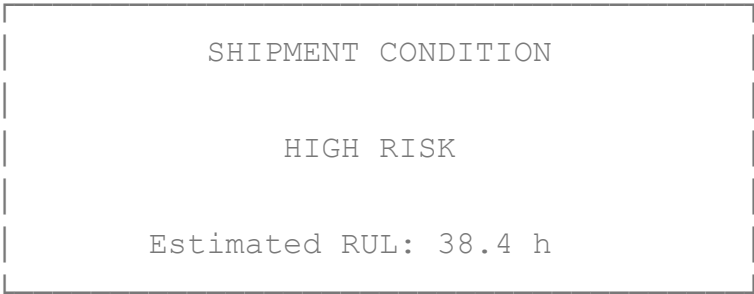
}

}

Phase 19 – Dashboard

The dashboard should not be a generic IoT graph.

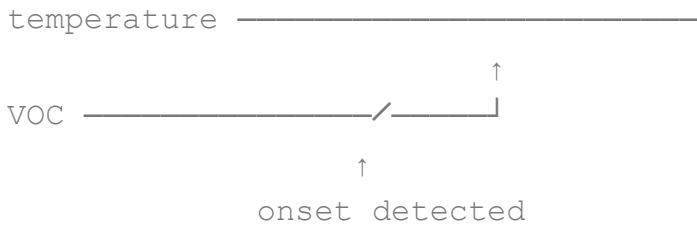
Main card



Risk decomposition



Timeline



Degradation velocity

stable → accelerating → rapid

Evidence timeline

```
14:02  baseline established
17:18  thermal excursion
18:31  VOC acceleration detected
18:42  biochemical onset detected
19:05  RUL threshold crossed
```

Phase 20 – Validation & Calibration

This phase is critical.

We should **not pretend the mathematical system is scientifically validated merely because the equations run.**

We need controlled validation eventually.

For each commodity:

```
fresh sample
  ↓
controlled storage
  ↓
continuous SGP41 + temperature + humidity
  ↓
periodic physical/quality assessment
  ↓
actual spoilage endpoint
```

Then evaluate:

Onset detection

```
predicted onset
vs
observed onset
```

Metrics:

- onset timing error
- false-positive rate
- false-negative rate

RUL

predicted remaining life
vs
observed remaining life

Metrics:

- MAE
- interval coverage
- early/late prediction error

Risk score

Eventually calibrate:

risk score → actual probability

Only after this should we confidently call it **spoilage probability** rather than a risk score.

Final project structure

I would now restructure Axios roughly as:

```
Axios/  
|  
├─ data/  
|   ├── raw/  
|   |   └─ telemetry/  
|   ├── processed/  
|   |   ├── telemetry/  
|   |   └─ sessions/  
|   └─ events/  
|  
└─ src/
```

```
| |— ingestion/
| |   |— mqttt.py
| |
| |— processing/
| |   |— filters.py
| |   |— baseline.py
| |   |— derivatives.py
| |   |— trajectory.py
| |
| |— biochemical/
| |   |— voc.py
| |   |— nox.py
| |   |— degradation.py
| |
| |— thermal/
| |   |— arrhenius.py
| |   |— exposure.py
| |
| |— detection/
| |   |— onset.py
| |
| |— rul/
| |   |— estimator.py
| |
| |— fusion/
| |   |— risk.py
| |
| |— evidence/
| |   |— events.py
| |
| |— api/
| |   |— routes.py
| |
|— artifacts/
|   |— configs/
|
|— scripts/
|   |— phase10/
|   |— phase11/
|   |— phase12/
|   |— phase13/
|   |— phase14/
|   |— phase15/
|   |— phase16/
```

```
|   |— phase17/
|   |— phase18/
|
|— dashboard/
|
|— reports/
```

What happens to Phases 1–9?

Keep them.

```
Phase 1–6 → Hohenheim data pipeline
Phase 7   → ML experiments
Phase 8   → ML evaluation
Phase 9   → ML model registry
```

But mark them:

Experimental / research branch – not used by production inference.

The new production branch starts at **Phase 10** and uses the actual SGP41 telemetry.

The five final outputs

Output	Initial implementation
Spoilage probability	Risk score → later calibrated probability
Spoilage onset	Signal-change + persistence + hysteresis detector
Remaining useful life	Degradation trajectory + thermal exposure → time-to-threshold
Degradation velocity	VOC/NOx first/second derivatives + trajectory state
Thermal vs biochemical risk	Arrhenius thermal burden vs VOC/NOx biochemical signal

This gives Axios a coherent engineering story without claiming that a model trained on 12 unrelated MQ channels can understand an SGP41. The Product X materials also explicitly emphasize VOC/NOx sensing, temperature/humidity, Arrhenius thermal exposure, spoilage onset, time-to-spoilage estimation, and an evidence record, so this revised architecture is aligned with that stated product direction.